NetworkPulse AI: An Evidence-Grounded Agentic Console for Network Intrusion Triage, Evaluated Beyond the Random Split

Sai Siddhanth
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
saisiddhanthks@outlook.com

Vihaan
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*—Machine-learning intrusion detectors trained on CICIDS2017 routinely report F1 scores above 99%, and large language models are increasingly proposed as analysts that explain their alerts. We present NetworkPulse AI, a working triage console that combines a weighted XGBoost–LightGBM–CatBoost ensemble, a benign-only autoencoder and Isolation Forest, a blended 0–99 risk score, and a tool-calling LLM agent that reads evidence only through seven read-only application tools and never acts on infrastructure. Its packaged benchmark reaches 99.88% F1, and we reproduce this ($99.91 \pm 0.02\%$) and show it survives validation-set thresholds, removal of train–test duplicates and a deduplicated natural-prior split. We then evaluate the same models on all 2.83 M flows under protocols closer to deployment. Trained on Monday–Wednesday and tested on Thursday–Friday, F1 falls to 22.2%, with precision 99.5% but recall 12.5%. Withholding one attack family at a time shows that the decision threshold, not the model, governs detection of novel attacks: a threshold set by a 1% benign false-positive budget recovers 68–99% of unseen DoS, DDoS, web and brute-force traffic that the F1-tuned threshold largely misses, while port scans and botnet traffic evade every detector. A claim-level audit of a deployed agent briefing finds every transcribed value correct but 10 of 36 claims misinterpreted, including a unit error and an invented attack correlation, and exposes a capped-score ranking failure. We report concrete fixes for each problem.**

***Index Terms*—intrusion detection, CICIDS2017, gradient boosting, anomaly detection, LLM agents, security operations, evaluation methodology**

I. Introduction

Security operations centres (SOCs) do not suffer from a shortage of alerts; they suffer from too many of them, and from alerts that arrive without the context an analyst needs to decide what to do next [1]. Machine-learning intrusion detectors trained on labelled flow data are usually presented as the answer to the first problem, and large language models (LLMs) are increasingly proposed as the answer to the second [2], [3]. Both claims deserve scrutiny. Flow classifiers evaluated on CICIDS2017 [4] routinely report F1 scores above 99%, yet the dataset has documented labelling and flow-construction problems [5], [6], and random train/test splits are known to overstate how well a detector will do on traffic it has not seen [7]–[9]. LLM analysts, for their part, write fluent explanations whose individual statements may or may not be true [10].

This paper describes *NetworkPulse AI*, a working intrusion-triage console that combines both ideas, and then asks how far each part of it can be trusted. NetworkPulse scores every flow with a weighted ensemble of three gradient-boosted tree classifiers [11]–[13], adds two detectors trained only on benign traffic (an autoencoder and an Isolation Forest [14]), blends the three signals into a 0–99 risk score, and hands a bounded evidence packet to a tool-calling LLM agent that writes an analyst briefing. The agent runs an *Observe* $\rightarrow$ *Detect* $\rightarrow$ *Inspect* $\rightarrow$ *Correlate* $\rightarrow$ *Reason* $\rightarrow$ *Recommend* loop, executes every tool inside the application rather than letting the model reach the network, and is forbidden from taking any action on infrastructure.

On its packaged benchmark—a stratified 80/20 split of a 120,000-flow sample—the ensemble reaches 99.88% F1. We show that this number is reproducible and survives the obvious within-sample corrections, and that it nevertheless says little about the question an operator cares about: will the detector flag an attack it was not trained on, recorded on a day it did not see? The contributions are:

1) **System.** A complete, open design for an evidence-grounded agentic triage console: a hybrid supervised/normal-only risk score, seven read-only evidence tools, a bounded tool-calling loop with a guaranteed final-synthesis path, and a deterministic local fallback (Section III).
2) **Evaluation beyond the random split.** On all 2.83 M CICIDS2017 flows we audit duplicates, then evaluate the exact NetworkPulse models under a cross-day split and a leave-one-attack-family-out protocol, measuring what the normal-only detectors add when the supervised ensemble meets an unseen family (Section IV, Section V).
3) **Agent audit.** A claim-by-claim verification of a briefing produced by the deployed agent against the evidence packet it was given, with an error taxonomy, and an analysis of a ranking failure in the risk score itself (Section VI).

We report every number from fixed seeds and state plainly where the system falls short; the shortfalls are, in our view, the most useful part of the paper.

II. Related Work

Flow-based intrusion detection. CICIDS2017 [4] provides five days of benign and attack traffic converted to bidirectional flows with roughly eighty statistical features, and has become one of the most used benchmarks in the area [15]. Gradient-boosted trees are a strong default for such tabular data [16]. Engelen *et al.* [5] and Lanvin *et al.* [6] showed that flow-construction and labelling errors in the released CSVs change measured detection performance, and Sarhan *et al.* [17] found that detectors trained on one dataset transfer poorly to another. We use the original CSVs that NetworkPulse ships with and treat these known problems as a reason to evaluate more strictly, not as something we fix.

Evaluation pitfalls. Sommer and Paxson [7] argued that intrusion detection is a poor fit for closed-world classification because the attacks that matter are the ones missing from the training data. Arp *et al.* [8] catalogued pitfalls—sampling bias, data snooping, inappropriate threshold selection—that inflate reported results in security ML, and Pendlebury *et al.* [9] showed that temporally inconsistent splits overstate malware-detector accuracy. Our cross-day and leave-one-family-out protocols follow directly from these papers.

Normal-only anomaly detection. Autoencoders trained on benign traffic flag inputs they reconstruct badly; Kitsune [18] is a well-known online example. Isolation Forest [14] scores points by how easily random partitions isolate them. Such detectors can in principle catch novel attacks, at the cost of more false alarms. NetworkPulse uses both as auxiliary evidence rather than as the primary detector; Section V measures whether that helps.

LLM agents for security operations. Tool-using language models interleave reasoning with calls to external functions [19], [20], and grounding generation in retrieved evidence reduces, but does not remove, unsupported statements [10], [21]. Surveys of LLMs in security [2] list alert triage and report writing among the most common proposed uses, and PentestGPT [3] showed both the promise and the reasoning failures of LLM agents on offensive tasks. We contribute a concrete, audited example from the defensive side.

III. The NetworkPulse AI System

Fig. 1 shows the architecture. A FastAPI back end loads the trained models and an evidence cache of scored flows, exposes them through a REST API, runs the agent, and serves a single-page analyst console (command centre, investigation deck, agent cockpit, model lab and reports). No component of the system can change network configuration: the agent's only outputs are text and recommended analyst actions.

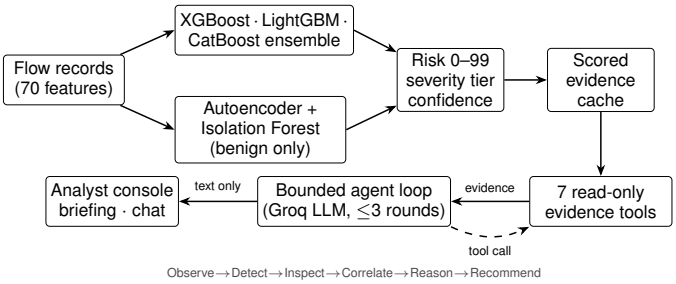


Fig. 1. NetworkPulse AI architecture. Detection evidence flows left to right; the LLM agent can read evidence only through application-side tools and can return only text.

A. Detection Layer

Features. Each flow is described by the 70 numeric CICIDS2017 features that remain after removing identifiers (flow ID, IP addresses, source port, timestamp) and constant columns; destination port is kept. Infinite values are treated as missing and imputed with training medians.

Supervised ensemble. XGBoost (400 trees, depth 7, learning rate 0.06), LightGBM (400 trees, 63 leaves, learning rate 0.05) and CatBoost (200 iterations, depth 8, learning rate 0.08) are trained as binary benign-versus-attack classifiers. Their probabilities are averaged with fixed weights,

$$p_{\text{ens}} = 0.5\,p_{\text{XGB}} + 0.3\,p_{\text{LGBM}} + 0.2\,p_{\text{Cat}}, \tag{1}$$

and a flow is labelled an attack when $p_{\text{ens}} \geq \tau$ ($\tau = 0.37$ in the packaged build). Agreement between the members is $a = \max(0, 1 - 2\sigma)$, where σ is the standard deviation of the three probabilities.

Normal-only evidence. A fully connected autoencoder (70–32–16–32–70, ReLU, Adam, early stopping) and an Isolation Forest (300 trees) are fitted to standardised benign flows only. The autoencoder's per-flow reconstruction error e is mapped to $s_{\text{AE}} = 100\,(1 - \exp(-e/1.5\,e_{97}))$, where e_{97} is the 97th percentile of benign training error; the Isolation Forest score is min–max scaled between the 2nd and 98th percentiles of the scored batch to give $s_{\text{IF}} \in [0, 100]$.

Risk and severity. The console ranks flows by

$$\begin{aligned} r &= \min\left(99,\ 100\,\tilde{r}\right), \\ \tilde{r} &= 0.82\,p_{\text{ens}} + 0.13\,s_{\text{AE}}/100 + 0.05\,s_{\text{IF}}/100, \end{aligned} \tag{2}$$

with tiers Critical ($r \geq 85$), High ($r \geq 65$) and Medium ($r \geq 40$). A confidence value $100\,[0.72\max(p_{\text{ens}}, 1-p_{\text{ens}}) + 0.28\,a]$ is shown beside each score. For a single flow the dossier also lists the ten largest XGBoost TreeSHAP contributions [22] and the ten features with the largest autoencoder reconstruction deviation.

B. Agent Layer

Evidence tools. The agent sees the data only through seven read-only functions: a network summary (counts, tiers, label distribution), the highest-risk events (filterable by severity, label or text), a single event dossier, related events (same label or destination port, nearest in risk), the held-out evaluation

metrics, the attack distribution, and global feature importance for any ensemble member. Tools are executed by the application, following the local tool-calling pattern, so the model never touches the data store or the network directly.

Two modes. In *autonomous* mode the back end runs one cycle every 180 s: it observes the cache, ranks the top six flows by r, inspects the top one, retrieves eight related flows, and sends this fixed evidence packet—with the summary and evaluation metrics—to the LLM in a single call that asks for a four-part briefing (what matters now; why the top flow is risky, separating ground-truth label from model evidence; correlated activity; three analyst-reviewable next steps). In *chat* mode the analyst asks a question and the model may call tools for up to three rounds of at most eight calls each; if the budget runs out, a final call with tools disabled forces a synthesis from the evidence already gathered, so the console never shows an empty answer. If the LLM is unreachable, a deterministic template summarises the top event from the same tools.

Guard rails. The system prompt instructs the model to work only from supplied evidence, never invent network facts, distinguish dataset labels from model predictions, and recommend only reviewable steps, never blocking, quarantining or reconfiguring anything. Generation uses temperature 0.15 and at most 1,400 tokens. Optional threat-intelligence lookups (VirusTotal, AbuseIPDB) are disabled unless the operator supplies keys, and the LLM credential is held server-side.

IV. Evaluation Protocols

All experiments use the original eight CICIDS2017 CSV files bundled with NetworkPulse (2,830,743 flows, Monday–Friday), the 70-feature set of Section III, and the exact model configurations of the build. Unless stated otherwise the seed is 42. Training ran on a two-core CPU machine; library versions are listed with the released code.

P1 – Packaged benchmark. The 120,000-flow evidence cache shipped with the build (60,000 benign, 60,000 attack, balanced by construction) is split 80/20 with stratification. As in the original training script, the decision threshold is the F1-optimal value on the *test* split. We repeat with seeds 42, 7 and 2026.

P2 – Within-sample corrections. Three changes address pitfalls listed by Arp *et al.* [8]: (a) the threshold is chosen on a validation split carved from the training data; (b) test flows whose feature vector also occurs in the training split are removed; (c) destination port, a plausible shortcut feature, is dropped.

P3 – Natural prior. Exact duplicate feature vectors are removed from the full corpus (first occurrence kept), and a stratified 500,000-flow sample with the corpus's natural attack share is split 68/12/20 into train, validation and test.

P4 – Cross-day. Models are trained on Monday–Wednesday (benign capped at 300,000 flows, each attack class at 40,000) and tested on all deduplicated Thursday and Friday flows. The threshold comes from an in-distribution validation split of the training days. Thursday and Friday contain web attacks, infiltration, botnet, port-scan and DDoS traffic, none of which occurs in the training days; the training days contain DoS, Heartbleed and FTP/SSH brute force.

TABLE I
Within-sample protocols on the 120k evidence cache (ensemble, mean ± s.d. over seeds 42, 7, 2026; %).

Protocol	F1	Recall	FPR
P1 packaged (test-set threshold)	99.91 ± 0.02	99.95 ± 0.03	0.13 ± 0.03
P2a validation threshold	99.90 ± 0.04	99.93 ± 0.04	0.13 ± 0.03
P2b + drop train/test overlap	99.89 ± 0.04	99.92 ± 0.04	0.13 ± 0.03
P2c validation thr., no dest. port	99.82 ± 0.04	99.87 ± 0.03	0.23 ± 0.04

P5 – Leave-one-family-out (LOFO). Attack labels are grouped into seven families: DoS (Hulk, GoldenEye, slowloris, Slowhttptest, Heartbleed), DDoS, PortScan, Brute-Force (FTP- and SSH-Patator), Web (brute force, XSS, SQL injection), Bot and Infiltration. For each family the supervised ensemble is retrained without it (other attack classes capped at 30,000 training flows each) on a fixed 300,000-flow benign sample split 60/20/20. The test set is the held-out benign flows plus *every* flow of the excluded family. Normal-only detectors are trained once on a 60,000-flow subsample of the benign training flows; they never see attacks, so the same models serve every fold.

Operating points. For P4 and P5 we report recall (the share of attack flows flagged) and false-positive rate (FPR) at thresholds fixed before looking at test data: the validation F1-optimal threshold for the ensemble; the threshold giving 1% FPR on benign validation flows for each detector; the console's own High tier ($r \geq 65$); and a simple OR rule that raises an alert when either the ensemble or the autoencoder exceeds its 0.5%-FPR threshold, so the combined false-alarm budget stays near 1%. We also report threshold-free ROC-AUC.

V. Results

A. The Packaged Number Is Real, and Robust Within the Sample

Table I reports P1 and P2. Re-training the build's models on the shipped cache reproduces the packaged score: 99.91 ± 0.02% F1 over three seeds, against 99.88% in the model card. Each of the three corrections moves F1 by less than 0.1 points. Choosing the threshold on validation data costs 0.01 points. About 1907 test flows per seed (8%) have an exact copy in the training split—the cache contains 8,062 duplicate feature vectors—but removing them changes F1 by 0.02 points. Dropping destination port costs 0.09 points and roughly doubles the false-positive rate. On the full corpus, 331,919 flows (11.7%) are exact duplicates and 719 distinct feature vectors carry more than one label (7,144 flows), consistent with the labelling problems reported in [5], [6]. After deduplication, a natural-prior random split (P3, 17% attacks) still gives 99.74% F1 at 0.058% FPR. Under random splitting, then, the detector is as good as claimed.

TABLE II
CROSS-DAY SPLIT (P4): TRAIN MON–WED, TEST ALL 987,188 DEDUPLICATED THU–FRI FLOWS (22.5% ATTACKS). THRESHOLDS FIXED ON TRAINING-DAY VALIDATION DATA EXCEPT WHERE MARKED.

Detector	F1	Prec.	Recall	FPR	AUC
Ensemble, same-day validation	99.96	99.94	99.97	0.01	1.000
Ensemble (τ=0.61)	22.2	99.5	12.5	0.02	0.773
Ensemble, test-tuned τ†	60.8	99.5	43.8	0.07	0.773
XGBoost alone	21.8	99.6	12.2	0.01	0.879
LightGBM alone	49.3	99.7	32.7	0.02	0.886
Autoencoder (1% val. FPR)	46.4	72.6	34.1	3.74	0.875
Isolation Forest (1% val. FPR)	37.6	73.2	25.3	2.69	0.781
Console High tier ($r \geq 65$)	22.7	99.6	12.8	0.02	0.904

†Uses test labels; an upper bound, not a deployable setting.

B. Across Days, Recall Collapses

Trained on Monday–Wednesday, the ensemble scores 99.96% F1 on held-out flows from those same days but only 22.2% F1 on Thursday and Friday (Table II). Precision stays at 99.5%: the model is not raising false alarms, it is silent. It flags 79–85% of web brute-force and XSS flows and 20% of DDoS flows, and essentially no port-scan (0.5%), botnet or infiltration flows. Lowering the threshold to the best value *on the test data*—information no operator has—raises F1 only to 60.8%. Threshold-free ROC-AUC falls to 0.773.

Two further observations matter for design. First, the weighted ensemble ranks unseen traffic worse than two of its own members (ROC-AUC 0.773 versus 0.879 for XGBoost and 0.886 for LightGBM), because CatBoost (0.695) pulls the average down; weights tuned on a random split do not transfer. Second, the autoencoder has the best threshold-free ranking of any single detector (ROC-AUC 0.875) and catches 34% of attack flows, including 59% of DDoS and 44% of infiltration flows. However, the threshold calibrated for 1% FPR on Monday–Wednesday benign traffic produces 3.7% FPR on Thursday–Friday benign traffic. Benign traffic drifts too. Blending the anomaly scores into the risk score does improve ranking (ROC-AUC 0.904 versus 0.773), but at the console's High tier this gain is invisible, because the tier is reached only when p_{ens} is already high.

C. Unseen Families: the Threshold Is the Lever

Fig. 2 and Table III report P5. With a family removed from training, the ensemble at its validation F1-optimal threshold misses most of it: 42% recall on DoS, 63% on DDoS, 73% on web attacks, and under 1% on brute force, port scan, botnet and infiltration. Yet in every fold it still *ranks* the unseen family well above benign traffic (ROC-AUC 0.83–0.998). The F1-optimal threshold (0.26–0.49) is tuned to the families the model has seen, which it scores near 1.0, and so it is far too high for anything new.

Setting the ensemble threshold for a 1% false-positive budget on benign validation traffic (thresholds of 0.003–0.007) recovers 96% of unseen DoS, 99% of DDoS and web attacks and 68% of brute force, at a measured benign test FPR of 0.95–1.14%. Port scans (3%) and botnet traffic (0%) remain

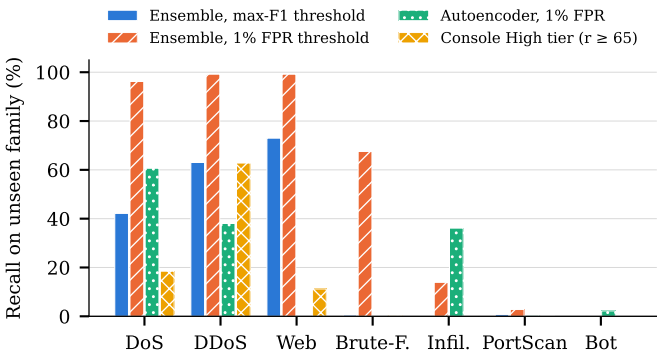


Fig. 2. Leave-one-family-out recall on the excluded family. The same trained ensemble catches most unseen DoS, DDoS, web and brute-force flows when its threshold is set by a 1% false-positive budget instead of by F1 on seen attacks. Benign test FPR is 0.06–0.13% for the max-F1 threshold and 0.9–1.1% for the 1%-budget detectors.

TABLE III
LEAVE-ONE-FAMILY-OUT (P5): RECALL (%) ON THE UNSEEN FAMILY AND THRESHOLD-FREE ROC-AUC. BENIGN TEST FPR ≈1% FOR THE FPR-BUDGET COLUMNS.

		Recall at fixed operating point				ROC-AUC	
Unseen family	n	Ens. max-F1	Ens. 1% FPR	AE 1% FPR	Risk $\geq$65	Ens.	AE
DoS	193,680	42.2	**96.2**	60.6	18.5	0.997	0.961
DDoS	128,013	63.0	**99.2**	38.0	62.8	0.998	0.847
Web	2,143	73.0	**99.2**	0.0	11.2	0.999	0.737
Brute-Force	9,152	0.5	**67.5**	0.0	0.2	0.984	0.691
Infiltration	36	0.0	13.9	**36.1**	0.0	0.891	0.939
PortScan	90,255	0.7	2.8	0.5	0.4	0.879	0.705
Bot	1,953	0.0	0.0	2.2	0.0	0.832	0.602

invisible: they resemble benign flows in this feature space, and every detector we tried, supervised or not, fails on them. The normal-only detectors help only where the supervised model has nothing to go on. On infiltration they catch 36–44% of flows against 14% for the ensemble, and they find 61–77% of unseen DoS. Elsewhere they are weaker than the low-threshold ensemble. The console's risk score of Eq. (2), whose High tier requires $p_{\text{ens}} \geq 0.79$ when the anomaly scores are low, inherits the high-threshold behaviour and flags only 0–63% of every unseen family. The OR rule (ensemble or autoencoder, 0.5% FPR each) does not beat the 1%-budget ensemble: it is lower on DoS, DDoS, web and brute force (94.7, 90.3, 99.0 and 24.5%) and only marginally higher on port scan, infiltration and botnet traffic (3.2, 41.7 and 1.6%), because halving the ensemble's budget costs more than the autoencoder adds.

VI. AUDITING THE AGENT

Detection metrics say nothing about whether the briefing an analyst reads is true. We therefore audited the most recent briefing written by the deployed agent (autonomous cycle 6, Groq back end, 613 words), checking each factual statement against the evidence packet the agent received and, where the packet was silent, against the dataset itself.

TABLE IV
CLAIM-LEVEL AUDIT OF ONE AUTONOMOUS BRIEFING (36 CLAIMS).

Category	n	Example from the briefing
Supported	26	Ensemble 0.9998; reconstruction error 15.21; 4 forward packets (26 B) vs. 3 backward (11,607 B)
Numeric error	2	"Flow duration ≈1.85 M µs (≈31 min)" (it is 1.85 s); "≥8 additional" related flows (7 plus the focal flow)
Semantic conflation	4	Dataset labels shown in a "Predicted label" column, although the detector is binary; agreement 0.99925 read as "99.9% of the three learners concur"
Unsupported inference	4	"Temporal clustering … a coordinated burst": the packet has no timestamps, and DoS Hulk and DDoS flows were recorded two days apart
Next steps	1/3	Output stopped mid-sentence at the 1,400-token limit

Procedure. We split the briefing into 36 atomic, checkable claims (a number, a label, a comparison or an inference) and assigned each to one category: *supported* by the packet; *numeric error*; *semantic conflation*, where a value is correct but described as something it is not; or *unsupported inference*, where the packet does not justify the statement or the data contradicts it. Recommendations were checked separately for completeness. One author performed the audit; the claim list and evidence packet will be released with the paper so that the audit can be repeated.

Findings. Table IV summarises the result. Every value the agent *copied* from the packet—probabilities, scores, packet counts, byte totals, inter-arrival times, event identifiers—was correct. All ten faulty claims arose when the agent *interpreted* those values. Four patterns stand out.

- *Unit arithmetic.* The flow duration was quoted correctly in microseconds and then converted to "≈31 minutes"; the true value is 1.85 s. A wrong duration of this size changes which attack hypotheses look plausible.
- *Label versus prediction.* Although the system prompt tells the agent to keep dataset ground truth separate from model evidence, the briefing presented the CICIDS2017 labels as model predictions twice. NetworkPulse's classifier is binary and never predicts "DDoS" or "DoS Hulk".
- *Invented correlation.* The related-event tool groups flows that share a destination port and have similar risk. Because many flows sit at the capped risk of 99 (see below), it returned DoS Hulk flows (Wednesday) next to DDoS flows (Friday); the agent described these as a single coordinated burst.
- *Sampling read as network state.* The evidence cache is balanced by construction (50% attacks). The agent read its 21.6% Critical share as evidence of "a pronounced attack surface focused on web services".

The briefing also ended mid-sentence after the first of three requested next steps because the response hit its token limit.

A ranking failure upstream of the agent. The audit exposed a problem in the risk score itself. Because Eq. (2) is capped at 99 and the boosted trees output probabilities indistinguishable from 1 for most volumetric attacks, 4,987 of the 120,000 cached flows (4.2%) share the maximum score. The "top six" flows that every autonomous cycle inspects are therefore an arbitrary pick from about five thousand ties, and the related-event search, which sorts by distance in risk, returns ties as well. In the logged run all six cycles inspected the same flow and produced near-identical briefings, because the cache does not change between cycles.

Implications. Grounding the agent in tool outputs did what it is meant to do for transcription and nothing for interpretation. Three inexpensive changes follow directly: (i) pass values to the model with units already converted and with label and prediction fields named unambiguously (`dataset_label`, `model_attack_probability`); (ii) rank and correlate on the uncapped logit of p_{ens}, with the anomaly score as tie-breaker, and add time and host keys to the related-event search; and (iii) check every number in a generated briefing against the packet before display and flag mismatches, which is cheap because the packet is structured. A single audited briefing is a case study, not an estimate of an error rate; Section VII returns to this limitation.

VII. DISCUSSION

What to report. A 99.9% F1 score on a random split of CICIDS2017 is reproducible, and it survives the usual within-sample corrections, but it measures how well the model recognises attacks it has already seen. The cross-day and leave-one-family-out results measure the harder question, and on that question the same models, with the same features, range from near-perfect to blind depending on the family. We suggest that papers on this dataset report at least one temporal or family-held-out split alongside the random one; the extra cost here was under an hour on a two-core machine.

Thresholds over models. The most actionable result is that the operating point, not the model, decided most of the unseen-family performance. A threshold chosen to maximise F1 on seen attacks sits where seen attacks end, and a new attack family does not score that high. A threshold set from a false-positive budget on benign traffic depends only on what normal traffic looks like, which is also what an operator can actually measure in deployment. The cost is a tenfold rise in benign alerts (from roughly 0.1% to 1%), which matters at SOC volumes [1]. That cost is exactly what the agent layer is meant to absorb, provided the agent's explanations can be trusted.

Role of the normal-only detectors. The autoencoder and Isolation Forest were added on the intuition that normal-only models catch the novel. They do so for infiltration and partly for DoS, and they rank cross-day traffic better than the supervised ensemble. As implemented, however, they are blended into a capped risk score whose High tier still demands a high supervised probability, so their contribution is mostly invisible to the analyst. We recommend surfacing them as a separate, budgeted alert stream and recalibrating their

thresholds on recent benign traffic, given the drift observed between days.

Limitations. All results come from one dataset, with its documented flow and label errors [5], [6]; we used the original CSVs to evaluate the system as built rather than a corrected release. Training sets were capped per class for run time, and the cross-day and family experiments use a single seed. The agent audit covers one briefing checked by one annotator, so it is a case study that motivates the fixes in Section VI, not an error-rate estimate; a larger study across many cycles, prompts and models, with two independent annotators, is the obvious next step. Finally, we evaluate detection and explanation, not analyst outcomes; a user study would be needed to show that the briefings shorten triage.

VIII. Conclusion

NetworkPulse AI combines a gradient-boosted ensemble, normal-only anomaly detectors and a bounded, tool-grounded LLM analyst into a working triage console. Its packaged 99.88% F1 is real but describes recognition of known attacks; across days, F1 falls to 22%, and when a family is withheld, recall depends far more on how the threshold is set than on which model is used. Setting the threshold from a 1% benign false-positive budget recovers 68–99% of four unseen families, while port scans and botnet traffic stay undetected by every model tested. The agent copied evidence faithfully but misinterpreted it in ten of 36 checked claims. Future work will apply the fixes identified here—budget-based thresholds, an uncapped ranking score, unit-normalised and unambiguously named evidence fields, and automatic numeric checking of briefings—and evaluate them across datasets and with practising analysts.

Reproducibility

All experiments use the eight original CICIDS2017 CSV files and the hyperparameters of the NetworkPulse build; seeds, scripts and the agent evidence packet used in the audit will be released with the camera-ready version.

References

[1] B. A. Alahmadi, L. Axon, and I. Martinovic, "99% false positives: A qualitative study of SOC analysts' perspectives on security alarms," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 2783–2800.

[2] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," arXiv:2405.04760, 2024.

[3] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symposium*, 2024, pp. 847–864.

[4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[5] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[6] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, ser. Lecture Notes in Computer Science, vol. 13857. Springer, 2023, pp. 18–33.

[7] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[8] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[9] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Security Symposium*, 2019, pp. 729–746.

[10] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 248:1–248:38, 2023.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[12] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[13] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 6638–6648.

[14] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[15] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[16] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.

[17] M. Sarhan, S. Layeghy, and M. Portmann, "Evaluating standard feature sets towards increased generalisability and explainability of ML-based network intrusion detection," *Big Data Research*, vol. 30, p. 100359, 2022.

[18] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[20] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[22] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.